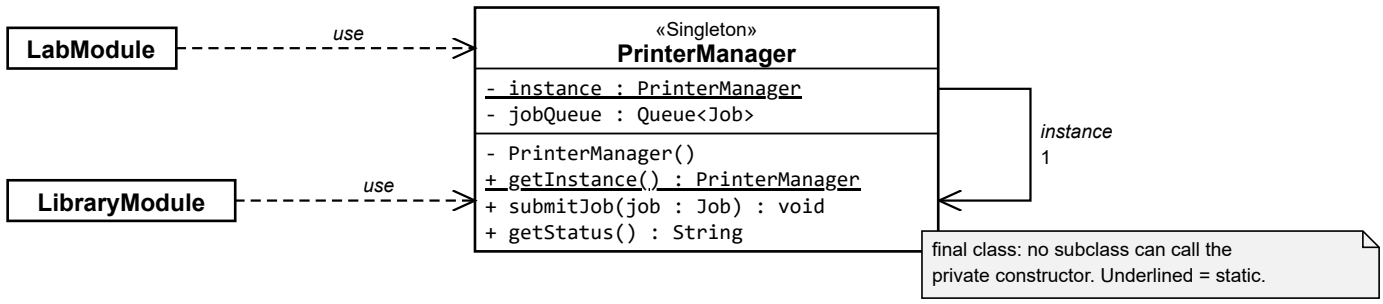
01 Singleton

Creational

Intent (GoF)	Ensure a class has **only one instance** and provide a **global point of access** to it.
Use when	One shared resource/manager must exist once for the whole system: printer queue, logger, DB connection manager, configuration, tax calculator, ID generator, payroll engine.
Trigger words	“only one”, “single global”, “same file / same queue”, “unique ID for all”, “run by only one user”, “new object created every time ⇒ duplicates / inconsistent status”. **“exactly / maximum N”** ⇒ **Multiton.**
3 ingredients	(1) **private constructor** (2) **private static** field holding the instance (3) **public static getInstance()**. Class `final` (Java) / `sealed` (C#).
Problems it removes	Memory waste · inconsistent data (each object has own state) · resource conflicts (wrong order, duplicates, freezes) · difficult management.

UML — Smart printing system (one PrinterManager for all departments)



Java

```
public final class PrinterManager {
    private static PrinterManager instance;        // the one object
    private final Queue<String> jobQueue = new LinkedList<>();

    private PrinterManager() { }                   // no "new" outside

    public static PrinterManager getInstance() {   // lazy
        if (instance == null)
            instance = new PrinterManager();
        return instance;                           // always same object
    }
    public synchronized void submitJob(String dept, String doc) {
        jobQueue.add(dept + ": " + doc);           // one FIFO queue
    }
    public String getStatus() { return jobQueue.size() + " jobs
waiting"; }
}

// client (any department)
PrinterManager p1 = PrinterManager.getInstance();  // creates obj#1
PrinterManager p2 = PrinterManager.getInstance();  // returns obj#1
System.out.println(p1 == p2);                      // true
```

Step	instance	Result
1st `getInstance()`	null → obj#1	object created, returned
2nd `getInstance()`	obj#1	no creation, same object
`p1 == p2`		**true** (same reference)
`new S(); new S();` (public ctor)		2 objects ⇒ **false**

Why?	Answer
private constructor	stops `new` from outside; class controls creation
static instance	belongs to the class ⇒ one copy shared by all
static getInstance()	no object exists yet to call a normal method on
final / sealed	a nested subclass could still reach the private ctor
volatile	all threads see the fully built instance (no reordering/caching)

Creation variants (thread safety)

```
// A. EAGER: thread-safe, created at class load
private static final Singleton INSTANCE = new Singleton();
public static Singleton getInstance() { return INSTANCE; }

// B. LAZY (above): NOT thread-safe. Two threads can both
//    see null and create two objects.

// C. synchronized method: safe, but locks on EVERY call
public static synchronized Singleton getInstance() {...}
```

```
// D. DOUBLE-CHECKED LOCKING: lazy + thread-safe + fast
private static volatile Singleton instance;
public static Singleton getInstance() {
    if (instance == null) {                    // check 1, no lock
        synchronized (Singleton.class) {       // C#: lock(obj)
            if (instance == null)              // check 2, locked
                instance = new Singleton();
        }
    }
    return instance;
}
```

Eager drawback (Sarcar): touching any static member (e.g. `Singleton.MyInt`) creates the instance even if never used. **Check 2** is needed because a second thread may be waiting at the lock while the first creates the object.

Extended Singleton = Multiton (N objects) · Object Pool (N reusable)

PrinterManager
- printers : List<PrinterManager> - MAX_PRINTERS : int = 3 - nextPrinter : int = 0
- PrinterManager() + getPrinter() : PrinterManager

Round robin: p1→Printer1 (new), p2→Printer2 (new), p3→Printer3 (new), p4→**Printer1** again.
1 DB manager ⇒ Singleton · 3 printers / 5 ATMs ⇒ Multiton · 10 reusable threads ⇒ Object Pool · unlimited customers ⇒ normal class.

```
public final class PrinterManager {
  private static final int MAX_PRINTERS = 3;
  private static final List<PrinterManager> printers = new ArrayList<>();
  private static int nextPrinter = 0;
  private PrinterManager() { }

  public static synchronized PrinterManager getPrinter() {
    if (printers.size() < MAX_PRINTERS)
      printers.add(new PrinterManager());          // create up to N
    PrinterManager p = printers.get(nextPrinter);
    nextPrinter = (nextPrinter + 1) % MAX_PRINTERS; // round robin
    return p;
  }
}
```

Advantages / disadvantages / challenge → mechanism

Advantages	Disadvantages	Challenge → mechanism to overcome
Saves memory (one object)	Hard to unit test (tight coupling, can't mock)	Race condition creates 2 objects → double-checked locking / eager init
Centralized control of shared resource	Hidden dependencies (global access)	Testing → Singleton implements an interface, inject it (DI) so tests pass a mock
Easy global access	Can become a global variable if overused	Bottleneck / single point of failure → Multiton / pool + standby replica (HA/DR)
No duplicate objects ⇒ consistent state	Lazy version not thread-safe	Subclass/reflection breaks it → `final`, private ctor. Overuse → only for true single resources

Questions & model answers

In this section: **Smart printing** SINGLETON · **One DB connection** SINGLETON · **Payroll - one user** SINGLETON · **Logger** SINGLETON · **Fantasy cricket** MULTITON · **Kiosk tax calculator** SINGLETON · **OrderID generator** SINGLETON

SINGLETON **SMART PRINTING** — Explain two problems that could occur because multiple PrinterManager objects are created. (2)

(1) **Inconsistent state:** each object holds its own queue/status, so departments see different printer statuses and no object knows the global order ⇒ jobs print in the wrong order. (2) **Resource conflict:** uncoordinated objects send jobs to the one printer simultaneously ⇒ duplicated jobs and the printer becomes unresponsive. (Also memory waste.)

SINGLETON **SMART PRINTING** — Recommend a design approach and explain why it is suitable. (3)

Singleton. Private constructor + private static instance + public static `getInstance()`; every department calls `PrinterManager.getInstance()` and gets the **same object** ⇒ one queue (FIFO order, no duplicates), one status for all, no wasted objects. Use double-checked locking because departments submit concurrently.

SINGLETON **ONE DB CONNECTION** — How do you ensure only one DB connection is active at a given time?

Make `DatabaseConnectionManager` a Singleton: private ctor, static `connection`; `getConnection()` opens it only if `null`, otherwise returns the existing one (synchronized). The Factory creates the right `Database` type once; everyone reuses that one connection.

SINGLETON **PAYROLL - ONE USER** — Payroll: salary calculation must run by only one user; all other connections/processes stopped. Pattern and why? (4)

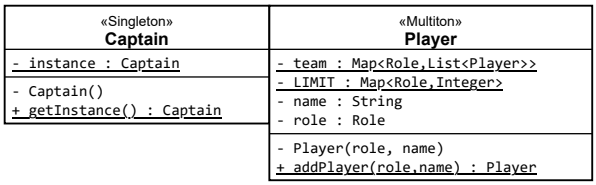
Singleton `SalaryCalculationManager` ⇒ only one calculation engine exists. Its `runPayroll()` is `synchronized` and sets a `running` flag; while set, other logins/processes are rejected or queued. Singleton = one controller; lock = one execution at a time.

```
public final class SalaryCalculationManager {
    private static volatile SalaryCalculationManager instance;
    private boolean running = false;
    private SalaryCalculationManager() { }
    public static SalaryCalculationManager getInstance() { /* double-checked locking (D) */ }
    public synchronized void runPayroll(String user) {
        if (running) throw new IllegalStateException("Payroll running - access blocked");
        running = true;  /* block other sessions, calculate */  running = false;
    }
}
```

SINGLETON **LOGGER** — Logger must write all logs to the same file; multiple instances cause duplicate/inconsistent output. Propose a pattern. (5)

Singleton Logger: one instance owns the single `FileWriter`; all classes call `Logger.getInstance().log(msg)`. One file handle (no corrupt/duplicate lines), one format and order, saves memory, global access. `log()` is `synchronized` so lines from threads don't interleave.

MULTITON **FANTASY CRICKET** — Fantasy cricket: max 5 batsmen, max 5 bowlers, 1 wicketkeeper, 1 captain — use an extended Singleton. (5)




```
public final class Player {
  private static final Map<String,Integer> LIMIT =
      Map.of("BATSMAN",5, "BOWLER",5, "WICKETKEEPER",1);
  private static final Map<String,List<Player>> team = new HashMap<>
();
  private final String role, name;
  private Player(String role, String name) { this.role=role;
this.name=name; }
  public static synchronized Player addPlayer(String role, String
name) {
    team.putIfAbsent(role, new ArrayList<>());
    if (team.get(role).size() >= LIMIT.get(role))
      throw new IllegalStateException("Max " + role + " reached");
    Player p = new Player(role, name); team.get(role).add(p); return
p;
  }
}  // Captain = plain Singleton (exactly one)
```

SINGLETON **KIOSK TAX CALCULATOR** — Single global tax calculator across all checkout counters — how is it integrated into the pricing flow? (4)

Decorated items give `item.getCost()`; in the **Payment** state the kiosk sums them and calls `TaxCalculator.getInstance().calculateTax(subtotal)`. Every counter gets the same instance ⇒ one rate, one rule set, consistent tax; change the rate once ⇒ changes everywhere.

SINGLETON **ORDERID GENERATOR** — Design a global OrderID generator and integrate it with payment processing. (5)

```
public final class OrderIdGenerator {
  private static final OrderIdGenerator INSTANCE = new
OrderIdGenerator();
  private final AtomicLong counter = new AtomicLong(1000);
  private OrderIdGenerator() { }
  public static OrderIdGenerator getInstance() { return
INSTANCE; }
  public String nextId() { return "ORD-" +
counter.incrementAndGet(); }
}
```

Eager Singleton + `AtomicLong` ⇒ thread-safe, never duplicates. In the template method `PaymentProcessor.handlePayment()` the first step is `id = OrderIdGenerator.getInstance().nextId()`, then validate → processPayment → sendReceipt(id). All payment subclasses share the same generator ⇒ unique IDs system-wide.

Combinations & mistakes

+ Factory	Factory chooses DB type; the connection manager (and often the factory) is a Singleton.
+ Facade	Only one facade object needed ⇒ make the facade a Singleton.
+ State	Stateless state objects (IdleState…) can be shared Singletons (vending machine slides).
Mistakes	Public constructor · non-static getInstance · not underlining static members in UML · claiming plain lazy is thread-safe · saying Singleton “blocks users” (it limits *objects*; blocking execution needs a lock too) · using Singleton for N objects (use Multiton) · calling it structural.